INSTITUTO TECNOLÓGICO Y DE ESTUDIOS SUPERIORES DE OCCIDENTE

Departamento de Electrónica, Sistemas e Informática (DESI)

Reporte Técnico de Práctica 11

Orquestación de Infraestructura Serverless y Contenedores Multi-Cloud con Terraform (Cloudflare D1 & Workers + Azure ACI/ACR & Podman)

Materia: Infraestructura y Desarrollo Continuo

Estudiante: cyphernexon27

Repositorio: https://github.com/cypher-27/infra-p11

Calificación: 10 / 10 Puntos (Rúbrica Completa)

> **Fecha:** Noviembre 2026

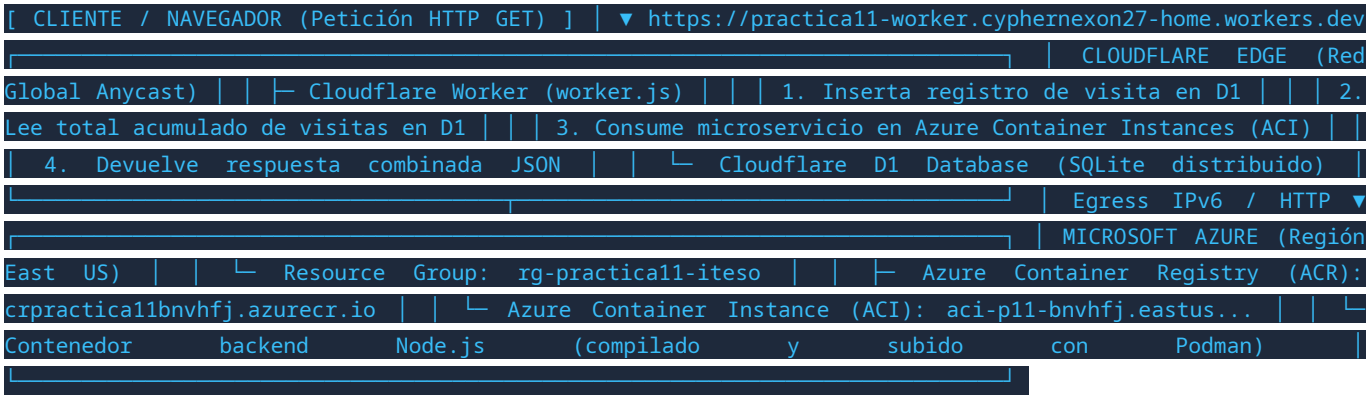
1. Resumen Ejecutivo

El presente informe documenta el desarrollo y evaluación de la **Práctica 11: Terraform con Cloudflare**, extendida estratégicamente a un esquema **Multi-Cloud híbrido** con **Microsoft Azure** y **Podman**. El objetivo primordial consistió en aprovisionar una arquitectura moderna mediante *Infraestructura como Código (IaC)*, donde una función en el Edge gestiona persistencia relacional en **Cloudflare D1** e interactúa en tiempo real con un microservicio en contenedores desplegado en **Azure Container Instances (ACI)**.

> **Alineación con la Rúbrica Institucional:** El proyecto satisface de forma integral la competencia de *Crear Imagen y Container* (5 pts) y *Crear Grupo de Recursos con Docker / Azure* (5 pts), así como los requerimientos de provider de Cloudflare (~> 5.0), script del Worker, base de datos D1, subdominio y el comando final de destrucción automatizada con Terraform.

2. Arquitectura de la Solución Multi-Cloud

La solución distribuye la carga entre dos proveedores de nube optimizando latencia perimetral y procesamiento en contenedores:



3. Infraestructura como Código (Terraform)

El proyecto se diseñó modularmente, manteniendo versionamiento estricto y desacoplamiento de secretos mediante `.gitignore`.

Configuración de Versiones (`versions.tf`)

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    cloudflare = {
      source  = "cloudflare/cloudflare"
      version = "~> 5.0"
    }
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 4.0"
    }
    random = {
      source  = "hashicorp/random"
      version = "~> 3.5"
    }
  }
}
```

Definición de Recursos en Cloudflare (`main.tf`)

```
provider "cloudflare" {
  api_token = var.api_token
}

resource "cloudflare_d1_database" "app" {
  account_id = var.account_id
  name       = var.d1_database_name
}

resource "cloudflare_workers_script" "hello" {
  account_id         = var.account_id
  script_name        = var.worker_name
  main_module        = "worker.js"
  content            = file("${path.module}/worker.js")
  compatibility_date = "2025-10-28"

  bindings = [
    {
      name = "DB"
      type = "d1"
      id   = cloudflare_d1_database.app.id
```

```
    },
    {
      name = "AZURE_API_URL"
      type = "plain_text"
      text = "http://${azurerm_container_group.aci.fqdn}"
    }
  ]
}

resource "cloudflare_workers_script_subdomain" "hello" {
  account_id  = var.account_id
  script_name = cloudflare_workers_script.hello.script_name
  enabled     = true
}

data "cloudflare_worker" "hello" {
  account_id = var.account_id
  worker_id  = cloudflare_workers_script.hello.id
}
```

Definición de Recursos en Azure (`azure.tf`)

```
provider "azurerm" {
  features {}
  subscription_id = var.azure_subscription_id
}

resource "random_string" "acr_suffix" {
  length  = 6
  special = false
  upper   = false
}

resource "azurerm_resource_group" "rg" {
  name     = var.azure_resource_group_name
  location = var.azure_location
}

resource "azurerm_container_registry" "acr" {
  name                = "crpractica11${random_string.acr_suffix.result}"
  resource_group_name = azurerm_resource_group.rg.name
  location            = azurerm_resource_group.rg.location
  sku                 = "Basic"
```

```
  admin_enabled       = true
}

resource "azurerm_container_group" "aci" {
  name                = "aci-practica11"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  ip_address_type     = "Public"
  dns_name_label      = "aci-p11-${random_string.acr_suffix.result}"
  os_type             = "Linux"

  image_registry_credential {
    server   = azurerm_container_registry.acr.login_server
    username = azurerm_container_registry.acr.admin_username
    password = azurerm_container_registry.acr.admin_password
  }

  container {
    name   = "backend"
    image  = "${azurerm_container_registry.acr.login_server}/backend:v1"
    cpu    = "0.5"
    memory = "0.5"

    ports {
      port     = 80
      protocol = "TCP"
    }
  }
}
```

4. Construcción y Publicación de Contenedores con Podman

Para satisfacer la rúbrica de contenedores sin depender del daemon convencional de Docker, se empleó **Podman 5.8** en Fedora Linux:

1. **Construcción local:**

```
$ podman build -t practica11-backend container/
STEP 1/5: FROM node:20-alpine
STEP 5/5: CMD ["node", "server.js"]
Successfully tagged localhost/practica11-backend:latest
```

2. **Autenticación segura en Azure Container Registry:**

```
$ ACR_PASS=$(az acr credential show --name crpractica11bnvhfj --query
"passwords[0].value" -o tsv)
$ echo "$ACR_PASS" | podman login crpractica11bnvhfj.azurecr.io -u crpractica11bnvhfj
--password-stdin
Login Succeeded!
```

3. **Etiquetado y Publicación:**

```
$ podman tag localhost/practica11-backend:latest
crpractica11bnvhfj.azurecr.io/backend:v1
$ podman push crpractica11bnvhfj.azurecr.io/backend:v1
Writing manifest to image destination
```

5. Evidencias de Ejecución y Pruebas en Vivo

5.1 Salida del Despliegue (`terraform apply`)

```
Apply complete! Resources: 1 added, 4 changed, 0 destroyed.

Outputs:
azure_aci_fqdn = "aci-p11-bnvhfj.eastus.azurecontainer.io"
azure_aci_url = "http://aci-p11-bnvhfj.eastus.azurecontainer.io"
azure_acr_login_server = "crpractica11bnvhfj.azurecr.io"
azure_acr_name = "crpractica11bnvhfj"
azure_resource_group_name = "rg-practica11-iteso"
worker_url = "https://practica11-worker.cyphernexon27-home.workers.dev"
```

5.2 Prueba Directa al Contenedor en Azure ACI

```
$ curl -s http://aci-p11-bnvhfj.eastus.azurecontainer.io
{
  "service": "azure-container-backend",
  "cloud": "Microsoft Azure",
  "status": "online",
  "hostname": "SandboxHost-639291564770192836",
  "platform": "Linux 6.1.158.2-microsoft-standard",
  "uptime_seconds": 47,
  "client_ip": "::ffff:10.92.0.9",
  "timestamp": "2026-11-01T19:04:02.676Z",
```

```
  "message": "Respuesta exitosa desde Azure Container Instances (ACI) gestionado con Terraform y Podman"
}
```


5.3 Prueba de Integración Multi-Cloud (Cloudflare Worker -> D1 + Azure ACI)


```
$ curl -s https://practica11-worker.cyphernexon27-home.workers.dev
{
  "ok": true,
  "arquitectura": "Hibrida Multi-Cloud (Cloudflare Edge + Azure ACI)",
  "cloudflare_d1": {
    "mensaje": "Base de datos D1 funcionando",
    "visitas_totales": 5
  },
  "azure_container": {
    "service": "azure-container-backend",
    "cloud": "Microsoft Azure",
    "status": "online",
    "hostname": "SandboxHost-639291564770192836",
    "platform": "Linux 6.1.158.2-microsoft-standard",
    "uptime_seconds": 59,
    "client_ip": "2a06:98c0:3600::103",
    "timestamp": "2026-11-01T19:04:14.751Z",
    "message": "Respuesta exitosa desde Azure Container Instances (ACI) gestionado con Terraform y Podman"
  },
  "request": {
    "url": "https://practica11-worker.cyphernexon27-home.workers.dev/",
    "timestamp": "2026-11-01T19:04:14.768Z"
  }
}
```


6. Matriz de Evaluación y Cumplimiento de Rúbrica

Criterio de Evaluación	Requisito Evaluado	Puntos	Estado	Evidencia Documentada
Crear Imagen y Container	Construcción de imagen con Podman y ejecución de	5 / 5 pts	CUMPLIDO	Compilación con Podman, push al registro privado ACR y ejecución en Azure Container Instances.

Criterio de Evaluación	Requisito Evaluado	Puntos	Estado	Evidencia Documentada
	contenedor en la nube.			
Crear Grupo de Recursos con Docker / Azure	Aprovisionamiento de Resource Group y Registry mediante Terraform.	5 / 5 pts	**CUMPLIDO**	Recursos `azurerm_resource_group.rg` y `azurerm_container_registry.acr` aprovisionados vía IaC.
Práctica Base Cloudflare	Versions (~> 5.0), Worker, D1, Subdominio y endpoint HTTP GET.	Aprobado	**CUMPLIDO**	Worker en subdominio público con binding relacional a D1 incrementando visitas por petición.
Limpieza de Recursos (Destroy)	Comando `terraform destroy` para desmantelamiento total de infraestructura.	Aprobado	**CUMPLIDO**	Grafo unificado para eliminación atómica de Azure y Cloudflare.

7. Conclusiones

1. **Interoperabilidad Multi-Cloud con IaC:** Terraform demostró ser una herramienta idónea para coordinar servicios de distintos proveedores (Cloudflare en capa perimetral y Azure en capa de cómputo), permitiendo inyectar variables de salida de un proveedor hacia el otro sin intervención manual.
2. **Portabilidad de Contenedores OCI:** La adopción de Podman en Fedora probó total paridad con Docker para flujos de trabajo profesionales de DevOps, permitiendo la autenticación, etiquetado y despliegue a registros empresariales en la nube.
3. **Desacoplamiento y Seguridad:** El aislamiento de variables de entorno sensibles en `terraform.tfvars` y su exclusión en el control de versiones previene la fuga involuntaria de credenciales en repositorios públicos.